CI/CD deployment to a Google Compute Engine VM

One-time GCP and GitHub setup, followed by each automated deployment.

ONE-TIME SETUP

01 Create GCP project and enable billing; enable Compute Engine, Artifact Registry, IAP, IAM, and Vertex AI APIs.

02 Create an Artifact Registry Docker repository.

03 Create GitHub deploy service account and Workload Identity Federation provider restricted to this repository.

04 Grant deploy identity image-push and VM access: Artifact Registry Writer, OS Login, IAP tunnel, and Compute Viewer.

05 Create VM runtime service account; grant Vertex AI access and Artifact Registry Reader.

06 Create VM; attach runtime service account and cloud-platform access scope.

07 Install Docker and gcloud on VM; configure IAP and WebSocket firewall access.

08 Create /etc/rag-chatbot/app.env on VM with app configuration and dependency endpoints.

09 Add required GCP and VM variables to GitHub production environment.

EACH DEPLOYMENT

01 Push or merge to main (or start workflow_dispatch manually).

02 GitHub Actions checks out the repository.

03 GitHub OIDC authenticates to GCP through Workload Identity Federation.

04 Build Docker image; .dockerignore excludes local secrets and data.

05 Tag image with commit SHA and push to Artifact Registry.

06 Copy deploy_vm.sh to the VM through IAP.

07 Run deployment script on the VM through IAP.

08 VM runtime identity pulls the new image from Artifact Registry.

09 Restart WebSocket gateway and retrieval router containers.

10 App calls Vertex AI using ADC from VM identity; no JSON key file.

The workflow deploys the gateway and retrieval router. It does not run ingestion on every release.

Setup checklist and deployment details

Complete these items before the first push to main.

GCP AND VM

1. Enable and create

- Enable Compute Engine, Artifact Registry, IAP, IAM, and Vertex AI APIs in the GCP project.
- Create an Artifact Registry Docker repository.
- Create the VM and attach a user-managed runtime service account with the cloud-platform scope.
- Install Docker and gcloud; permit IAP SSH and WebSocket traffic through the firewall.

2. Assign separate identities

- VM runtime account: Vertex AI access and Artifact Registry Reader.
- GitHub deploy account: Artifact Registry Writer, Compute Viewer, OS Login, and IAP tunnel access. Grant serviceAccountUser only where needed to log in as the VM identity.

3. Configure VM runtime settings

- Create /etc/rag-chatbot/app.env with project and region, Kafka, Elasticsearch, Redis, Ollama, JWT, and application settings.
- Keep the file restricted to administrators. Do not place a service-account JSON key in it.

GITHUB AND RELEASE

4. Configure GitHub

- Create a production environment and add these variables:
- GCP_PROJECT_ID; GCP_WORKLOAD_IDENTITY_PROVIDER; GCP_DEPLOY_SERVICE_ACCOUNT;
- GAR_LOCATION; GAR_REPOSITORY; GCE_VM_NAME; GCE_VM_ZONE.
- Restrict Workload Identity Federation to this repository and intended branch/environment.

5. Release flow

- Push or merge to main. GitHub builds an image tagged with the commit SHA and pushes it to Artifact Registry.
- GitHub connects to the VM through IAP, copies the deploy script, and invokes it.
- The VM pulls the image with its attached identity and restarts the WebSocket gateway and retrieval router.
- Use workflow_dispatch to run the deployment manually.

SCOPE AND SECURITY

Ingestion is not run automatically on release. Rotate any credentials previously exposed in the tracked .env file before production.

Single-VM deployment restarts containers during release and is not highly available.